Web Application Security Part 2: XSS Analysis Lab Penetration Testing Report Template

Name: [Humphrey Wanjala]
Date: [Date]
Role: Junior Penetration Tester

Engagement Summary

Instructions**:** Write a concise summary of your testing session. Include:

- The goal of this engagement (testing Stored and Reflected XSS under Medium and Impossible security)
- What vulnerabilities you attempted
- What outcomes were observed
- One sentence on the significance of security level differences

The goal of this engagement was to see how many of the same payloads would work at the walkthroughs that had been done in the previous blog post for Stored and Reflected XSS at the Low security setting could work at the Medium and Impossible security settings and what mitigations would stop them from working if any security settings were changed. The Guestbook (Stored XSS) and name parameter (Reflected XSS) modules were re-tested with <script>alert('XSS')</script> and case/tag variations. At Medium the payload was blocked literally but the filter itself was not complete, and could be bypassed by changing the casing or the syntax of the event-handler. All of the payloads tested at Impossible were rendered harmless, no matter the variation. The exercise shows that a security level is in essence the level of input validation and output encoding that is done: the medium security level is a naive blacklist filtering, while the impossible security level is a thorough input validation and output encoding done per context.

Testing Process Overview

Instructions: Briefly describe how you approached the lab:

- Which DVWA modules you tested
- How you changed security levels
- Tools used (Burp Suite, DevTools, source viewer)
- How you confirmed whether a payload executed or not

The tests were run on the Medium and Impossible security levels using the following modules: DVWA Stored XSS (Guestbook) and Reflected XSS (name parameter), and SetUp/Reset DB was used to reset the database between tests. Intercepting the form submissions using Burp

Suite Community Edition (in Browser) enabled the inspection of the request parameters before and after the server-side processing by DVWA, and allowed for confirmation of encoding/stripping were happening at the time of parameter submission or at the time of rendering. The rendered DOM/page source was inspected using Firefox DevTools after submission to see if the payload was stripped, shown as text (but not executed), or was executed (alert fired). For each level, the DVWA's built-in Source Code Viewer (found in the bottom-right "View Source" link on each vulnerability page) was used to view PHP handling logic.

Findings Table

Instructions**:** Fill in the table below for each test you performed.

Module	Security Level	Payload Used	Result (Executed, Encoded, Blocked)	Notes on Behavior / Output
Stored XSS	Low	<script>alert('XSS')</script>	Executed	Guestbook shows alert popup
Stored XSS	Medium	<script>alert('XSS')</script>	Blocked	<script> string stripped via str_replace(); message posts but tag content is gone
Stored XSS	Medium	<ScRiPt>alert('XSS')</ScRiPt>	Executed	Case-variant bypasses the case-sensitive str_replace('<script>', ...) filter
Reflected XSS	Low	<script>alert('XSS')</script>	Encoded	Rendered as literal text (<script>...) via htmlspecialchars(); no execution regardless of case or tag variation

Reflected XSS	Medium	<script>alert('XSS')</script>	Executed	name parameter echoed directly into page, alert fires
Reflected XSS	Impossible	<script>alert('XSS')</script>	Encoded	htmlspecialchars() on output neutralizes all tested variants, including event-handler payloads

Note: (Add more rows if you test additional payloads or variations.)

Mitigation Analysis

Instructions: Choose one example (Stored or Reflected XSS at Medium or Impossible) where the payload **did not execute**.

Answer the following:

- What did the output look like?

  The payload was visible as a literal string of text on the page, not as markup: <script>alert('XSS')</script> was not evaluated as markup, but as a string of text. No event was fired in the dialog and when I checked the HTML source code, it showed < and > instead of < and >.

- What code or filter appeared to block the script?

  Unlike Medium's str_replace("<script>", "", $_GET['name']), source/impossible/xss_r/source/index.php does not escape the name GET parameter with htmlspecialchars() before echoing it to the page.

- What security function(s) were used (e.g., htmlspecialchars())?

  htmlspecialchars(), which converts special HTML characters (<, >, &, ", ') into their corresponding HTML entities.

- Was the mitigation applied during input or output?

This mitigation is applied at output — the raw input is still accepted and stored/reflected as-is, but it is encoded at the point of rendering, right before it's written into the HTML response.

- Why was this approach effective (or not)?

  This is because output encoding is content agnostic, and doesn't attempt to detect or blocklist any "dangerous" string (which is why Medium's str_replace('<script>', '') can be circumvented by case changes, alternate tags, or event handlers, for example). Rather, it ensures that no matter what the browser receives in that context, it will never be able to be interpreted as executable markup language, no matter what the attacker sends. This is in line with the general secure-coding principle of output encoding, as appropriate to the rendering context, is a more secure control than input blacklisting.

Reflection

Instructions: Respond briefly:

- How did the behavior differ from what you saw in the walkthrough?
- What surprised you about DVWA's handling of different security levels?

The more useful distinction from the walkthrough was the fact that the exact same payload as in the walkthrough failed at Medium level, but succeeded at Low level, which could easily be interpreted as "the vulnerability is fixed" if testing were to stop at that point. It wasn't — the filter was only able to match a literal, case-sensitive <script> substring, so trivial variations (case changes, <img onerror=...>, <svg onload=...>) were still able to work. The lesson to be learned is that if a payload fails, it isn't because a class of vulnerabilities has been closed, it is because that particular string was matched by that particular filter. I was surprised the the Medium filter is so simple in the source — just a single str_replace(), not a whacky 'we added an XSS sanitization' section of code — and that this is actually a good example of something that looks like a 'sanitizing' filter in a changelog but doesn't actually do a lot of sanitizing in the real world. In the future I need to think twice before assuming "blocked" and "not vulnerable" are the same thing and instead check if it is blacklist based (try variants) or encoding based (try and confirm if the encoding covers the actual render context).